Collaborator Briefing

Reliability of LLM-Based Technical Interview Evaluation

Agreement, consistency, and style bias in rubric-based grading

Everything a new team member needs to know, from the first idea to where the project stands today (9 October 2026).

How to use this document. Section 1 gives you the whole project in one page. Sections 2 to 9 explain how we got here, what we are doing, and why. Sections 10 to 12 cover status, risks, how to help, and a glossary. Please do not share the answer key or the raters' files (Sections 8 and 9), and read the note on blind rating before you touch any data.

1. The project in one page

What we study. Many tools (interview-practice apps, online exams, hiring research) now ask a large language model (LLM) to grade a candidate's answer. We ask: **how reliable is an LLM as a grader of technical interview answers?** Specifically, do its scores agree with human raters, are they stable when the same answer is graded again, and are they pushed around by how an answer is written (long, formal, ungrammatical) rather than by whether it is technically right?

How we study it. We built a controlled benchmark: 40 technical questions, each with 8 versions of an answer. Some are correct, some partial, some wrong; some are plain, some wordy, some ungrammatical, some formal and polished. Because we wrote the versions, we know what changed between them. We then (a) ask several LLM "judges" to score all 320 answers under a fixed rubric, several times, and (b) have independent human raters score a 100-answer subset without knowing which version is which. Then we compare.

Why it matters. If a judge gives a wrong but polished answer a higher score than a correct but plainly written one, practice tools mislead learners and hiring tools may treat candidates unfairly. We want evidence, not assumptions, about when LLM grading can be trusted.

Where we are. Literature review done (16 papers read). Design and protocol written. Dataset of 320 answers built, checked by script, and label-reviewed (ready to freeze). Blind human rating is finished: 4 raters scored a 100-answer subset with strong agreement. Judge-runner and analysis scripts are built and tested (the analysis on simulated and fake data only). The introduction of the paper is drafted. The real judge run and the analysis of real scores have not been done yet. Roughly 40 to 45 percent of the total work is done; the biggest remaining items are the real run, the analysis, and the writing.

Item	Number
Questions	40 (7 computing domains)
Answer versions per question	8
Total answers	320
LLM judges (current)	2 (a third, stronger one is optional)
Judge scoring conditions per answer	7
Planned judge calls	4,480 (for 2 judges)
Human-rated subset	100 answers, 4 independent raters (finished)

2. How we got here (the story so far)

The project was developed in a long working conversation with an AI assistant (Claude), with a parallel conversation in ChatGPT for second opinions. Here is the sequence, because several decisions only make sense in context.

Stage	What happened	Why it matters
Topic 1 (dropped)	A first topic was proposed: dependency hallucination in LLMs (do they recommend packages that do not exist?). A critique found it crowded (package-hallucination or "slopsquatting" work already exists), partly trivial (package lookups), and with a weak novelty story.	We changed topic before spending time on it.
Topic 2 (chosen)	Reliability of LLM judges for technical interview answers. Chosen because it is feasible solo or in a small team (API calls, no GPU), ethical (synthetic data, practice setting), objective enough to measure, and relevant to GenAI careers. It builds on the lead author's interview-preparation platform.	Two alternates were noted: citation faithfulness in multi-agent research systems, and stability of SHAP explanations in injury prediction.

Stage	What happened	Why it matters
First literature search	Two quick searches showed judge biases are well studied for pairwise comparisons, rubric-based judges work well on dialogue but poorly on factual or domain-heavy text, and no peer-reviewed study of technical interview scoring surfaced.	Gave a first version of the gap.
Step 1: literature review	16 papers downloaded and read (Section 4). The closest is a Journal of Applied Psychology paper on LLM scoring of employment interviews (behavioral constructs, not technical correctness).	Sharpened the gap and the overlap risk.
Step 2: protocol v0.1	A written protocol: research question, hypotheses, 12-variant design (3 quality levels x 4 styles), rubrics, judge prompt, metrics, ethics. 20 database questions drafted.	Later partly superseded by the larger dataset (Section 5).
Tooling	A Python script (run_judges.py) that sends each answer to several judges under fixed conditions and saves every score. Dry-run tested, then a tiny real API test passed.	Experiments can be run as soon as the dataset is frozen.
ChatGPT dataset v0.1	ChatGPT produced a 240-answer candidate (40 questions x 6 variants). We checked it and found the manipulations did not work: the "verbose" answers were shorter than the concise ones, the "partial" answers were cut off mid-sentence, the "incorrect + polished" ones were neither polished nor subtle, and the IDs and rating sheet leaked the labels.	Do not freeze a dataset without testing that its manipulations actually hold. We rebuilt it.
Dataset v0.2	Rebuilt in four batches of 10 questions, 8 variants each (320 answers), with random IDs, a separate secret key, a blind rating sheet, and a script that checks the manipulations.	Current candidate dataset (not yet frozen).
Human rating	A stratified 100-answer subset and two blind rating workbooks were created. Four raters finished. They agree strongly (ordinal Krippendorff alpha 0.945), mainly because most answers are clearly right or wrong.	Gives the human reference for agreement analysis.
Label review and scripts	All 160 partial and incorrect labels were reviewed against the humans' scores; only two answers (Q24 S6 and S8) were rewritten and 14 borderline partial answers were flagged. The analysis and freeze scripts were written and tested; a consistency test needed extra temperature-0 repeats, so the judge runner now makes 7 calls per answer per judge.	Dataset ready to freeze; real run pending.
Paper draft	Introduction written in a five-part structure (background, evidence, gap, objective, scope), plus a skeleton of the whole paper and 17 references.	Section II (literature table) is next.

3. Research questions and hypotheses

Main question. How reliably do LLM judges score technical interview answers against a fixed rubric, and how do answer style, rubric wording, and repeated runs change those scores when technical content is held constant?

RQ	Question	What we measure
RQ1 Agreement	How closely do LLM scores match human raters?	Quadratic weighted kappa, ICC, exact and adjacent agreement, mean absolute error
RQ2 Consistency	How stable are scores across repeated runs?	Run-to-run ICC, standard deviation, share of answers whose score moves by 1 or more points
RQ3 Style bias	Does presentation style change scores when content is equivalent?	Paired score differences between style variants of the same question
RQ4 Rubric sensitivity	Does rewording the rubric change scores or agreement?	Rubric A versus Rubric B differences

RQ	Question	What we measure
RQ5 Cross-judge	Do different judges behave differently?	All of the above, per judge

Hypotheses (provisional thresholds, to be frozen before running the real judges).

- **H1:** Under Rubric A each judge reaches weighted kappa of at least 0.60 with human consensus.
- **H2:** At default temperature at least 10% of answers show a score range of 1 point or more across 3 runs, more than at temperature 0.
- **H3:** Among correct answers, the polished jargon-heavy version scores higher, and the poor-grammar version lower, than the concise version.
- **H4:** Among correct answers, the verbose version scores higher than the concise version.
- **H5:** Switching from Rubric A to Rubric B changes scores and agreement.
- **H6 (key risk):** Incorrect but polished answers (S6) score higher than incorrect plain answers (S8). Partial confident (S5) versus partial plain (S7) is the matching check. A sanity check: incorrect answers should score below correct ones.

Note: the protocol document planned a fully crossed design (3 quality levels x 4 styles). The final dataset has 8 variants instead (Section 5), so style can be compared at the correct level in four styles, and at the partial and incorrect levels in two styles each.

4. What the literature says (16 papers read)

We read papers on LLM-as-a-judge bias, rubric-based evaluation, and automated grading. Key points:

Paper (short)	What it found or did	Use for us
Zheng et al. 2023 (MT-Bench)	Introduced LLM-as-a-judge benchmarks; named position and verbosity bias.	Foundation; terminology
Wataoka et al. 2024	Pairwise study: GPT-4 showed the strongest self-preference; judges favored lower-perplexity text.	Reason not to use the same LLM family to write answers and judge them
Ye et al. 2024 (CALM)	Framework quantifying 12 bias types; biases persist in advanced models.	Bias taxonomy
Chen et al. 2024	Humans and LLM judges are both biased (fake references, length, etc.).	Human raters are not perfect either
Gao et al. 2026	Pointwise scoring, 11 bias types, 6 judges: injected factual errors cut scores sharply (e.g. GPT-4o 9.12 to 4.00); verbosity had little effect under a detailed rubric.	Positive-control idea; evidence that verbosity effects are mixed
Soumik 2026 (TMLR)	Style bias (favoring markdown over plain prose) far larger than position bias; verbosity bias differs by model; some debiasing strategies help.	Supports testing style, not just position
Siro et al. 2026	LLM-designed rubrics: correlation with humans about 0.8-0.9 on dialogue, about 0.5 on news summaries, below 0.3 on biomedical summaries.	Reliability drops where facts matter
Hong 2026 (RULERS); Shen 2026 (RRD); Rao 2026 (Autorubric); Pan 2026 (RubricEval)	Methods and tools for better rubric judging: locked rubrics with evidence, recursive rubric refinement, an open framework with psychometric metrics, and a benchmark showing rubric-level judging is far from reliable.	Mitigation ideas and tooling
Choi et al. 2026 (IRT)	Uses Item Response Theory to separate a judge's intrinsic consistency from human alignment.	Method for ordinal scores
Fandina et al. 2025 (REFINE)	Tests judges by whether they rank outputs of known quality in the right order; improved alignment from below 0.7 to above 0.9 in some tasks.	Why we build answers of known quality

Paper (short)	What it found or did	Use for us
Mello et al. 2025 (LAK)	Short-answer grading: traditional models beat GPT-4o; useful prompt parts differed by language. Human agreement itself was modest.	Human ceiling may be low
Zhu et al. 2025	On real enterprise exams, zero-shot LLM baselines reached QWK of about 0.36-0.38 versus about 0.87-0.88 on LLM-generated data.	Evidence that technical, real data is harder
Stockdale, Hickman and Liu (J. Applied Psychology, 2026)	Psychometric study of LLM scoring of employment interviews: ensembles of larger models can match or beat single human raters; lower temperature helps; LLM scores sometimes show bigger group differences.	Closest paper; covers behavioral constructs, not technical correctness
Two papers set aside	An interview-prep platform paper (no validation of its scoring; references we could not verify) and a LeetCode code-solving paper (not about grading).	Not cited

The research gap

- The closest study (Stockdale et al.) covers behavioral interview constructs in employment selection and says generalization is unknown.
- Most bias studies use pairwise comparison or inject bias into good answers, without human scores for technical answers of varying correctness.
- Evidence on verbosity and style is mixed, and we found no study crossing answer quality with style while holding technical content fixed.
- Agreement, consistency, and rubric sensitivity are usually studied separately.

These statements reflect the studies we reviewed, not an exhaustive survey. A Google Scholar and arXiv check by the team is still needed before the paper claims novelty.

5. The benchmark dataset (v0.2 candidate)

Contents. 40 questions across seven domains: DSA 7, DBMS 7, OS 6, Networks 5, OOP and software engineering 5, AI/ML 5, Cloud and distributed systems 5. Difficulty: 14 easy, 16 medium, 10 hard. Every question has 8 answer versions (320 answers). All questions are short, definitional or conceptual ("What is X?"), which is a stated limitation.

Code	Quality	Style	What changes from S1	Average words
S1	Correct	Concise (baseline)	-	25
S2	Correct	Verbose	About 3x longer: filler and restatement, no new facts	76
S3	Correct	Poor grammar	Same facts, ungrammatical or non-native phrasing	26
S4	Correct	Polished, jargon-heavy	Same facts, formal vocabulary	25
S5	Partial	Confident	Misses one key concept, confident tone	30
S6	Incorrect	Polished	Fluent, formal, plausible but wrong	35
S7	Partial	Plain	Same omission as S5, plain tone	18
S8	Incorrect	Plain	Same error as S6, plain wording	24

S5 and S7 are a matched pair (same missing concept), as are S6 and S8 (same error). This lets us ask whether polish alone changes the score of an answer of fixed quality.

Example: all 8 versions for one question

Question: What is the time complexity of binary search?

Version	Answer text
S1	Binary search runs in O(log n) time. It requires a sorted array and halves the remaining search range with every comparison, so the number of steps grows logarithmically with the input size.
S2	That's a good question. Binary search is a searching algorithm, and its time complexity is O(log n), which is logarithmic time. To explain why: it works on a sorted array, and at each step it looks at the middle element and compares it with the target. Based on that comparison, it throws away one half of the remaining range, so the range shrinks by half each time. Because the search range keeps being halved, the number of steps needed grows only logarithmically with the input size. So, in summary, to answer your question directly, the time complexity of binary search is O(log n), as it halves the sorted search range at every step.
S3	Binary search have O(log n) time complexity. It need the array to be sorted, and every comparison it cut the remaining range to half, so steps is growing logarithmic with input size.
S4	Binary search attains logarithmic asymptotic time complexity, O(log n). Operating on a totally ordered array, each comparison bisects the candidate interval, so the number of probes is bounded by the logarithm of the input cardinality.
S5	Binary search is an extremely efficient algorithm because every comparison eliminates half of the remaining search space. That is the key idea behind it, and it is why it is far faster than checking elements one by one.
S6	Binary search has a time complexity of O(n log n). The array must first be sorted, which costs O(n log n), and the subsequent search over the ordered data proceeds by repeated halving, so the overall bound is dominated by the sorting phase.
S7	Binary search is faster than checking elements one by one because each comparison removes half of the remaining search space.
S8	Binary search has a time complexity of O(n log n). The array has to be sorted first, which costs O(n log n), and then the search uses halving, so the total is O(n log n).

How the dataset was made and checked

- **Authorship:** the answers were drafted by an LLM (Claude), based on the question list from the earlier ChatGPT candidate, and are marked "drafted, pending human review". The paper must describe this honestly (LLM-assisted, human-verified).
- **Manipulation checks:** a script (check_manipulations.py) confirms that the verbose answers are at least 2.5x the length of the concise ones, that answers end in complete sentences, that no question has duplicate answers, that IDs do not reveal labels, and more. Hard checks pass; soft warnings remain because the incorrect-polished answers (S6) are longer than S1 in many questions (about 35 versus 25 words on average). Word count must be a covariate in the analysis.
- **Blinding:** answer IDs are random. Labels live only in a separate answer key.
- **Label review:** a worksheet lists all 160 partial and incorrect answers next to a correct reference so a person can verify that partial answers really miss something and incorrect answers really contain a central error. It was done after blind rating: all 160 labels were confirmed, two Q24 answers were rewritten, and 14 borderline answers were flagged (humans scored them nearly correct or nearly empty).
- **Freezing:** after review the files are renamed answers_final and key_final, a file fingerprint is recorded, and no answer is edited based on judge scores.

Known weaknesses: single-LLM authorship, short definitional questions, S6 longer than S1, S5 longer than S7 (about 30 versus 18 words), and the earlier summaries of the batch 2 and 3 reviews appear to have been written by another AI and do not replace a human read-through.

6. The judges, the rubrics, and the script

Rubric A (weighted): "Score the answer on three criteria: Correctness (40%): are the technical claims accurate? Explanation (30%): is the reasoning clear and logically ordered? Completeness (30%): are the key concepts and important cases covered? Combine them into one 0-5 score."

Rubric B (prioritized): "Judge primarily whether the answer is technically correct. Give secondary consideration to the clarity of the explanation and to whether the key concepts are covered. Give one 0-5 score."

Score anchors (both rubrics): 5 fully correct, key concepts present, precise terms. 4 correct with a minor omission. 3 mostly correct but missing an important concept or with one minor error. 2 partial, significant gaps or one major error. 1 mostly incorrect, fragments of knowledge. 0 incorrect, irrelevant, or empty.

Judge prompt (identical for every judge and condition):

```
You are a computer science instructor grading a candidate's answer in a technical
interview.
Question: {question}
Key concepts: {expected_concepts}
Rubric: {rubric}
Candidate answer: {answer}
Return only JSON: {"score": <integer 0-5>, "justification": "<max 2 sentences>"}
```

The wording was changed from "database instructor" to "computer science instructor" because the benchmark covers seven domains; it must now stay fixed. The score comes first in the JSON so the judge does not reason before scoring. Deliberately, the prompt does **not** tell the judge to ignore style: an anti-bias instruction would hide the bias we want to measure, and is reserved as a later mitigation experiment.

Judge runs per answer, per judge (7 conditions)

Rubric	Temperature	Runs
A	0	3
B	0	1
A	provider default	3

Current judges: a Google Gemini flash-lite model and OpenAI gpt-oss-20b served through Groq (the script key is named groq_gptoss; the exact model IDs are set in the script and must be reported in the paper with run dates). Both are small models, so adding a larger third judge is recommended to see whether size matters. Planned volume: 320 answers x 2 judges x 7 conditions = 4,480 calls (6,720 with three judges). The three temperature-0 repeats let us compare run-to-run variation at temperature 0 against the default temperature (hypothesis H2).

The script, in plain words

- It reads the answers CSV, shuffles the order, sends each answer to each judge under each condition, parses the JSON score, and appends one CSV row per call.
- It retries on rate limits, logs failures, and resumes safely if stopped: rerun the same command.
- A --dry-run option produces fake scores without calling any API (use it to test; delete fake output files afterwards).
- API keys live in environment variables, never in files or chats.

7. Analysis plan (script built and tested on fake data)

Fixed before data collection to avoid choosing methods after seeing results:

Question	Metrics and tests
Agreement	Quadratic weighted kappa, ordinal Krippendorff's alpha, ICC, exact and adjacent agreement, mean absolute error; Pearson and Spearman as supplements. Reported relative to the human-human ceiling.
Consistency	ICC across runs, standard deviation per answer, share of answers with score range of 1 or more.
Style and verbosity	Paired differences within quality level, Wilcoxon signed-rank with Holm correction, effect sizes, bootstrap confidence intervals.
Joint model	Mixed-effects model: score ~ quality x style + log(word count) + (1 | question) + (1 | judge).
Rubric sensitivity	Mean absolute difference between Rubric A and B; change in kappa versus humans.
Human data	Use every rater who completes all 100 answers. Exclude someone only for objective reasons (incomplete, identical score on every row, admitted AI use) and report it. Human reference score = median across raters (mean for correlations). Report ordinal Krippendorff's alpha across all raters and weighted kappa for each pair. Do not pick raters by how well they agree.

Decision rule after the pilot: scale up only if human-human weighted kappa is at least 0.60 and at least one style effect is measurable; otherwise revise the rubric anchors and re-pilot.

8. Human rating: rules you must know

- **Subset:** 100 answers, 12 or 13 per variant (50 correct, 25 partial, 25 incorrect), each question appearing 2 or 3 times, and no question containing both members of a matched pair (S5 with S7, S6 with S8). Domains and difficulty are spread (Easy 34, Medium 40, Hard 26).
- **Result:** four raters finished. Ordinal Krippendorff alpha 0.945; mean pairwise weighted kappa 0.949; exact agreement 74% overall but only 36% on the 24 mid-range answers. Humans scored every correct answer about 5 whatever its style, incorrect answers near 0 (polished or plain), and partial answers about 3.
- **Two workbooks (A and B)** contain the same 100 answers in different random orders, with instructions, the rubric, validated 0-5 score cells, and a progress counter. Raters see no variant labels.
- **Rater rules:** work alone, no AI tools, look up facts if needed, no discussion until everyone finishes, expect 60 to 90 minutes.
- **Record keeping:** save returned files as R1_A.xlsx, R2_B.xlsx, etc., and log date, time taken, and whether facts were looked up.
- **Order matters:** raters score first. Only afterwards does anyone open the answer key or do the label review. Whoever rates blind must not have seen the key or this briefing's dataset examples.

Important for you, as a new collaborator: because you will read this briefing (including the labeled example in Section 5), please do **not** act as a blind rater. Better roles are listed in Section 11.

9. Files and how to run them

File	What it is	Share?
run_judges.py	Runs the LLM judges and saves scores (7 calls per answer per judge)	Yes (no keys inside)
analyze_results.py	Builds all result tables and a chart from the judge scores	Yes
freeze_dataset.py, check_manipulations.py, apply_review.py	Lock the dataset, test the manipulations, apply label-review edits	Yes
data/answers_v0.2_reviewed.csv	The 320 reviewed answers for the judges (random IDs, no labels)	Yes
data/answer_key_v0.2_reviewed.csv	Labels per answer, plus human_rated and review_flag columns	Only after you finished any blind rating
data/label_review_S5-S8_REVIEWED_v2.csv	Reviewed label sheet (verdicts, notes, flags, the 2 edits)	Yes
human_rating/human_ratings_merged_4raters.csv	Scores from the four raters	Yes
docs/ (protocol, paper draft, this briefing)	Project documents	Yes
COMMANDS.md, README.md	All commands in order, with expected outputs	Yes

Every command, in order, with the output to expect, is in COMMANDS.md (setup, dataset check, dry run, freeze, key setup, real run, analysis). Before any real run, the dataset must be frozen and delete any fake or test output files so they never mix with real scores.

10. Status and what is left

Phase	Status	Notes
Topic and literature review	Done	16 papers read
Protocol and design	Done	Needs one update to reflect the 8-variant design
Dataset (320 answers)	Reviewed, ready to freeze	All 160 hard labels reviewed; 2 answers rewritten; 14 flagged
Human rating	Done	4 raters, strong agreement
Judge script	Built and tested	Real run waits for the freeze and API keys
Analysis script	Built and tested on simulated data	Not yet run on real judge scores
Paper	Introduction drafted	Literature table, methodology next; results and discussion need data
Revisions and submission	Not started	Choose venue and format (IEEE style likely)

Suggested order from here

- Freeze the dataset (final files plus fingerprint).
- Run the real judges (consider adding a third, stronger judge).
- Run the analysis script on the real scores; read the style-bias and agreement tables first.
- Write results, discussion, conclusion; finish the literature table and methodology; write the abstract last.

11. Risks, honest limitations, and how you can help

Risk or limitation	What we do about it
Answers drafted by one LLM; may be more regular than real candidates and may favor judges from the same family	Disclose; add some real human-written answers later; compare judges from different families
Questions are short and definitional	State as a limitation; add applied questions in future work
S6 is longer than S1; S5 longer than S7	Use word count as a covariate; discuss
Small, cheap judges	Add a larger third judge; report exact model IDs and dates
Humans score nearly all correct answers 5 (ceiling effect) and the middle of the scale is thin	Style bias on correct answers can only show as scores below 5; lean on the partial and incorrect pairs; report mid-range agreement separately
Novelty claim based on a limited search	Do a Scholar and arXiv check; keep claims hedged
Many references are preprints	Verify venues, years, and pages before submission
AI assistance in building the benchmark and drafting	Follow the target venue's disclosure policy; keep a log of human verification

Ways you could contribute

- **Independent label reviewer:** re-check the 14 flagged answers and the two Q24 rewrites (not blind, so not a rater).
- **Literature owner:** write Section II and its comparison table; verify every reference and run the novelty check.
- **Analysis:** review or extend the analysis script once it exists, and recompute key numbers independently.
- **Replication:** run the dry run and checks on your own machine to confirm everything works.
- **Third judge and costs:** choose and configure a stronger judge within API limits.

Authorship, roles, and workload are for the team to agree between themselves.

12. Glossary

Term	Meaning
LLM-as-a-judge	Using a language model to grade or compare outputs.
Pointwise scoring	Scoring one answer at a time (as opposed to choosing the better of two).
Rubric	Written grading criteria and score anchors given to the judge.
Style bias	Score changes caused by presentation (length, grammar, formality) when content is unchanged.
Verbosity bias	Tendency to favor longer answers.
Self-preference bias	A judge favoring text written by itself or a similar model.
Temperature	Randomness setting; 0 is the most deterministic.
Default temperature	Provider default; scores can vary between runs.
Quadratic weighted kappa (QWK)	Agreement statistic for ordinal scores that penalizes big disagreements more than small ones; 1 is perfect, 0 is chance.
ICC	Intraclass correlation: how consistent scores are across raters or repeated runs.
Krippendorff's alpha	Agreement measure that works with several raters and ordinal scores.
Holm correction	Method to control false positives when running several statistical tests.
Mixed-effects model	Regression that accounts for repeated measures on the same questions and judges.
Dry run	Running the script with fake scores to test it without calling any API.
Freezing the dataset	Locking the answer file (with a fingerprint) so it cannot change after seeing results.
Blind rating	Rating without knowing which version or quality label an answer has.
Preprint	A paper posted online (for example arXiv) before or without peer review.

Questions to ask the lead author on day one: Are the judge model IDs final? Has the dataset been frozen? What is the target venue and deadline? What role would you like to take?